



Innovative Applications of O.R.

Optimizing automated sorting in warehouses: The minimum order spread sequencing problem[☆]

Nils Boysen^{*}, Stefan Fedtke, Felix Weidinger

Friedrich-Schiller-Universität Jena, Lehrstuhl für Operations Management, Carl-Zeiß-Straße 3, Jena, 07743, Germany



ARTICLE INFO

Article history:

Received 14 March 2017

Accepted 21 March 2018

Available online 28 March 2018

Keywords:

Facility logistics

Warehousing

Order merging

Sequencing

ABSTRACT

In warehouses, order consolidation processes are inevitable whenever picking orders are assembled under a zoning and/or batching policy. In our specific warehouse setting, bins containing partial orders picked under a zoning and batching policy are intermediately stored in an automated storage/retrieval system (ASRS) and, afterwards, released on a conveyor system supplying the consolidation area. Here, a fully-automated sorter merges partial orders dedicated to the same customer in packing stations where, finally, the customers' cardboard boxes are packed. For sequencing the release of bins from the ASRS we derive an elementary optimization problem, which aims to minimize the spread of orders in the release sequence so that picking orders are quickly assembled at their packing stations. The resulting problem is formalized, computational complexity is proven, and efficient solution procedures are provided. Additionally, managerial aspects are addressed with the help of a comprehensive simulation study.

© 2018 Elsevier B.V. All rights reserved.

1. Introduction

In the e-commerce era, many supply chains promise next-day or even same-day deliveries to their customers (e.g. Yaman, Karasan, & Kara, 2012). This puts increasing stress on the warehousing processes and necessitates a fast and efficient retrieval of stock keeping units (SKUs) from their shelves. Both in the warehousing literature and in practical applications especially the following two policies have been identified to considerably accelerate the retrieval process in picker-to-parts systems (see de Koster, Le-Duc, & Roodbergen, 2007):

- **Zoning:** Instead of picking each customer order in a unique tour, under the zoning policy the warehouse is partitioned into disjoint zones. Order pickers are fixedly assigned to zones and only pick the part of an order that is stored in their assigned zone. This way, the picking process is parallelized and order pickers only traverse smaller areas of the warehouse.

- **Batching:** A further reduction of unproductive travel time can be achieved by unifying multiple orders to batches that are jointly retrieved in a tour. This requires the identification of orders that are similar with respect to their due dates and the proximity of their storage locations.

The main disadvantage of both policies is that they both require a subsequent order consolidation. Zoning requires the merging of multiple partial orders picked in different zones and batching necessitates the separation of different orders jointly picked. In most warehouses, this task is executed by fully-automated sorters. Thus, care has to be taken that the pros of both policies, e.g., reduced picker travel, are not outweighed by the additional effort for the order consolidation process. While most scientific research is dedicated to the former part of this trade-off (e.g., see the survey papers of de Koster et al., 2007 and Gu, Goetschalckx, & McGinnis, 2007; Gu, Goetschalckx, & McGinnis, 2010), this paper focuses on the order consolidation process. Specifically, we investigate an elementary optimization problem, which aims to avoid congestion on the sorter. The resulting minimum order spread sequencing problem is inspired by the warehousing processes at a large fulfillment center of one of Germany's largest online retailers for shoes and clothing. However, the warehouse setting described in the following section is general enough so that our minimum order spread sequencing problem should also be relevant for other warehouses having to efficiently consolidate picking orders.

[☆] This research has been supported by the German Science Foundation (DFG) through the grant "Planning and operating sortation conveyor systems" (BO 3148/5-1).

^{*} Corresponding author.

E-mail addresses: nil.boysen@uni-jena.de (N. Boysen), stefan.fedtke@uni-jena.de (S. Fedtke), felix.weidinger@uni-jena.de (F. Weidinger).

URL: <http://www.om.uni-jena.de> (N. Boysen)

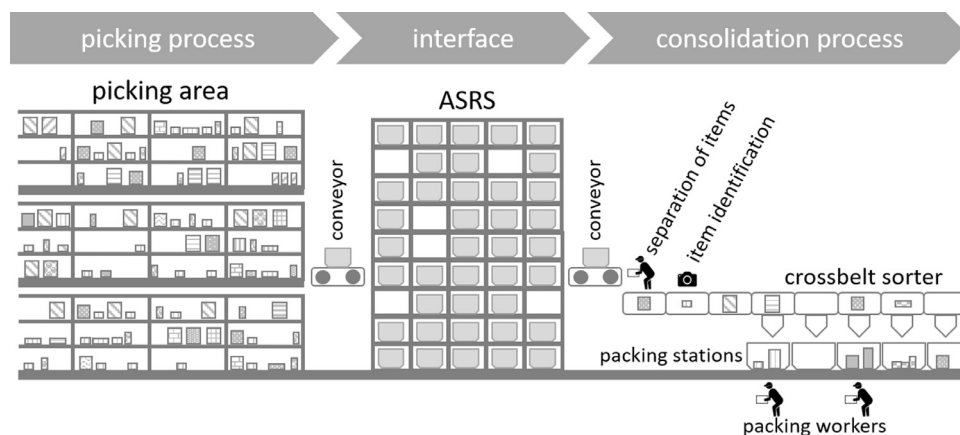


Fig. 1. Schematic layout of the warehouse setting.

1.1. The picking and order consolidation process at a large online retailer

After the arrival of inbound trucks, our online retailer scans and isolates all received items and individually stores them according to a chaotic, shared storage policy (for a detailed discussion of both aspects, e.g., see (Bartholdi & Hackman, 2014; Weidinger & Boysen, 2017)). Individual items are randomly scattered all over the shelves of the picking area consisting of about man-high racks stacked on top of each other in a mezzanine system (see Fig. 1). As the assortment of goods offered by our online retailer is wide, the vast picking area is partitioned into different zones. Furthermore, as is common in the business-to-consumer segment (e.g., Gallien & Weber, 2010) the number of items per order line is small so that orders are batched and a combined zoning and batching policy is applied. An order picker receives a pick list, which due to the zoning and batching policy refers to partial orders of multiple customer orders, on his/her handheld and picks the defined items into a bin manually pushed on some picking cart. A completed bin is put onto a conveyor system, which has multiple access points all over the picking area and leads to an automated storage/retrieval system (ASRS). The ASRS intermediately stores the bins and, thus, builds the interface between the picking and the subsequent order consolidation process.

Once all bins referring to a subset of customer orders (denoted as a wave, see de Koster et al., 2007) have arrived in the ASRS the order consolidation process is initiated. The respective bins are successively retrieved from the ASRS and placed on an accumulation belt. The conveyor delivers the bins towards an induction station where human workers deplete the bins and isolate the items on separate segments of a crossbelt sorter. Note that in alternative warehouse settings also fully-automated induction technology (see Gallien & Weber, 2010) and alternative sorting systems, e.g., bomb-bay and tilt tray sorters (see Briskorn, Emde, & Boysen, 2017; Johnson & Meller, 2002), can be applied. After passing a recognition station, where the bar code is scanned to identify each item, the belt moves along a sequence of successive packing stations (also denoted as lanes). Once the station currently assigned to the customer order of the present item is reached the sorter is initiated and moves the item via a gravity chute to the respective packing station. As soon as all items of a customer order are collected, a green beacon next to the station indicates the new status. Afterwards, a packing worker moves to the packing station and fills the collected items into a cardboard box, which is moved onwards to the shipping area by another conveyor. This releases the packing station in such a way that it can be assigned to a subsequent customer order.

The paper on hand treats the question for the right sequence, in which a given wave of bins containing a certain subset of customer orders should be released from the ASRS. Our online retailer faces the problem that especially during peak-hours a large workforce is required at the packing stations to timely release the lanes. Otherwise there may not be enough free lanes for collecting successive orders of subsequent waves and the sorter runs into what Gallien and Weber (2010) call a gridlock (see Section 1.2). In this paper, we aim to reduce the required workforce in an indirect manner. We formulate a surrogate objective, which aims to minimize the order spread (also denoted as order-consolidation time Owyong & Yih, 2006). For a given sequence, the spread of an order is defined by the number of sorter segments from the very first occurrence of an item dedicated to an order up to its last occurrence and we aim to minimize the sum of all order spreads. This way, all orders are collected in a shorter time span so that there is more flexibility for the packing workers to move from station to station. In a simulation study, we test whether our surrogate objective is well-suited to achieve the actual objective of our retailer, i.e., a reduced workforce.

Note that our research is tailored towards the warehouse setup of our online retailer. However, the methods developed here are also applicable in related warehouse settings. The ASRS from where the bins are retrieved need not be a high-bay rack, but can also be some carousel rack (see Litvak & Vlasiov, 2010) or puzzle-based storage system (e.g., Gue & Kim, 2007) and also a manual order consolidation process should profit from optimized bin release sequences.

1.2. Literature review

Applying a zoning and/or batching policy in a distribution center intends to reduce the travel distances when retrieving picking orders from the shelves. This part of the trade-off is well-researched and extensively surveyed in the review papers of de Koster et al. (2007) and Gu et al. (2007, 2010). The successive consolidation process has received less attention and is in the focus of the paper on hand, so that we restrict our review on papers integrating automated sorters.

On a strategic level, the question which environment justifies the application of an automated sorter is addressed by Petersen (2000) and Russell and Meller (2003). During the design phase, different layout aspects and control parameters are evaluated by analytical models and simulation studies. Bozer and Sharp (1985) investigate the impact of the number of packing stations on throughput. The impact of the waves, e.g., their degree of overlapping, is investigated by Bozer, Quiroz, and Sharp (1988). Johnson

and Lofgren (1994) report on the successful application of a sorter simulation during the setup of a new warehouse and Johnson and Meller (2002) predict the performance of induction stations with an analytical model.

Another part of the relevant literature deals with operational problems impacting the consolidation process of automated sorters. The interdependency between the number of zones and the performance of the consolidation process is investigated in (de Koster, Le-Duc, & Zaerpour, 2012). Meller (1997) defines an integer programming model for deciding on the assignment of customer orders to packing stations, if the items already circulate on a closed-loop sorter. For the same setting, Johnson (1998) develops an analytical model approximating the impact of varying lane assignment policies on expected total sorting time. Only two papers focus the blocking of packing stations. Owyong and Yih (2006) address the formation of batches in a relatively simple warehouse setting along a single straight conveyor belt. A batching and zoning policy is applied and pick lists are to be generated, such that the expected order-consolidation time is minimized. A heuristic procedure denoted as push algorithm is presented. Gallien and Weber (2010) consider waveless picking in the picking area where incoming orders are directly added to the order queues in the zones of a warehouse. They model the impact of the queue length on congestions in the packing stations.

All the literature just discussed treats a different setup of order consolidation. They consider warehouses where the batches collected in the picking area are put onto a conveyor directly leading to the automated sorter. Here, the loop conveyor itself is applied as intermediate buffer until all items of a customer order have arrived and a packing station can be assigned. The sorter acts as both the intermediate storage device and the means of sorting. In our warehouse setting, these two functions are separated and an extra ASRS takes over intermediate storage. There are two main reasons for applying an intermediate ASRS:

- Whenever there is a very large number of partial orders to be handled, a direct induction requires a large sorter having enough storage capacity for all items. A larger capacity increases the investment into the loop conveyor and requires plenty of space on the shop floor. In such a setting, an extra ASRS may be the better choice, because a large number of bins can be stored in a more compact way without requiring excessive space.
- A direct induction comes along with the risk of gridlocks. Whenever all stations are still occupied, newly arriving orders cannot be assigned a lane and in the worst case accumulating upstream congestion necessitates costly and laborious recovery procedures (see Gallien & Weber, 2010). This risk can be avoided if a wave-picking approach is applied. If the number of orders forming a wave does not exceed the number of packing stations, then waves just have to be released successively, such that all packing stations are unoccupied before the next wave arrives. However, such a sequential policy leads to a low utilization of the sorter and inefficient picking process due to an unavoidable unbalance of the pickers' workload (Petersen, 2000). Against this drawback waves are often released in an overlapping manner. Once a specific completion threshold is reached, e.g., Bozer et al. (1988) suggest 90% and Johnson and Lofgren (1994) report of 50%, the next wave is released. This, again, bears the risk of blockings if waves are released too early. Thus, a direct induction always requires a delicate steering between gridlocks and efficient capacity utilization. An intermediate ASRS, however, serves as a buffer between the consolidation and picking process. Order pickers do not need to wait for others to complete their batches until a wave can be released. They can directly proceed with successive batches, which are

intermediately stored in the ASRS. Once all batches are collected in the ASRS, waves of orders not exceeding the number of available lanes can successively be released in direct succession. This way, gridlocks are avoided with certainty provided that the workforce of packing workers is able to empty the lanes fast enough. To support the latter this paper introduces the minimum order spread sequencing problem.

Sequencing of items to be retrieved from and stored in an ASRS has a long lasting tradition in scientific literature. These research efforts are summarized in the in-depth survey papers provided by Roodbergen and Vis (2009) as well as Boysen and Stephan (2016). The focus of this stream of literature is, however, on reducing the effort for the storage and retrieval machine (also denoted as crane) that moves the items between their storage positions and the input/output point. Our sequencing approach presupposes that the crane is not the bottleneck, but rather targets an efficient consolidation process. Furthermore, sequencing of the shelf visits when retrieving orders in a traditional picker-to-parts system is a vividly discussed issue and order sequencing is also an important decision problem during order batching. These sequencing approaches aim to reduce the picker travel and, thus, have a completely different objective. Instead of trying to summarize that vast literature on these aspects we refer to the in-depth survey papers of de Koster et al. (2007) and Gu et al. (2007, 2010).

In the end, it can be concluded that an order consolidation process including an intermediate ASRS has not been treated in the literature yet.

1.3. Contribution and paper structure

This paper formulates the minimum order spread sequencing problem, which optimizes the retrieval sequence of bins released from an ASRS, such that the picking orders are quickly collected at the packing stations. The problem is described and modeled in Section 2. Appropriate exact and heuristic solution procedures based on dynamic programming are presented in Section 3. Then, the efficiency of these algorithms is tested in a computational study (Section 3.4). Furthermore, these algorithms are integrated into a warehouse simulation, which evaluates the impact of the retrieval sequence on the occupation of packing stations and the work schedules of packing workers (Section 4.3). Finally, Section 5 concludes the paper.

2. The minimum order spread sequencing problem

Once all bins containing the partial orders of a special subset of customer orders have arrived in the ASRS, the wave of bins is complete and can be released from the ASRS towards the consolidation area. We seek a suited release sequence for the given wave J of bins. Each bin $j \in J$ contains multiple partial orders referring to the set of customer orders O , which are to be accumulated in the packing stations. Bins containing order o are addressed by set $J_o \subseteq J$. The bin content is specified by parameters b_{jo} , which define the number of units in bin j dedicated to customer order o . The total number of items per bin is given by $B_j = \sum_{o \in O} b_{jo} \forall j \in J$.

A bin released from the ASRS is, first, transported towards the induction station where human workers or an automated mechanism successively deplete the bin and isolate each item on a separate belt segment of the automated sorter. The items of bin $j \in J$ are separated onto B_j successive belt segments and, then, directly succeeded by the next bin j' depleted onto the next $B_{j'}$ segments. We, thus, presuppose that empty belt segments, e.g., due to late arrivals of bins or human related delays, have only negligible impact and can, thus, be excluded. The order spread s_o of an order $o \in O$ is defined by the number of belt segments from the segment

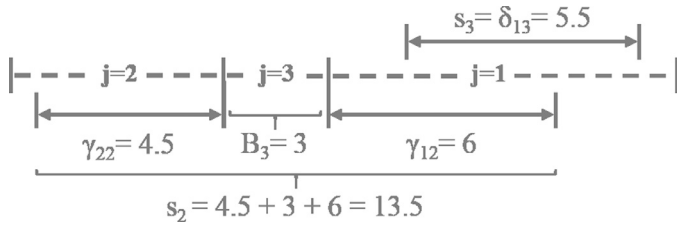


Fig. 2. Order spread example.

the very first item of order o is located on to the last segment of this order. As items belonging to the same order are typically spread over multiple bins, the order spread clearly depends on the bin sequence. The more bins with large content are sequenced in between the first and last occurrence of an order, the longer its order spread. Furthermore, it is also the detailed induction sequence of a bin, which influences the order spread. Specifically, an order's first and last occurrence depends on the induction sequence of the first and last bin containing an item for o , respectively. However, as the bins' detailed packing plans are not stored in the central information system, accurately predicting this sequence seems hardly possible. Without any further information the bin depletion process equals a random sampling without replacement. Thus, the expected number γ_{jo} of crossbelt segments passing by (if bin j is the first in sequence among those carrying items for order o) from the first occurrence of an item for o up to the last item of bin j is negatively hypergeometric distributed and amounts to (e.g. (Prokhorov, 2002))

$$\gamma_{jo} = \frac{b_{jo} \cdot (B_j + 1)}{b_{jo} + 1} \quad \forall j \in J; o \in O. \quad (1)$$

The same expected number γ_{jo} of sorter segments passes by from the very first item withdrawn from bin j until the last item dedicated to order o is depleted from j , which is relevant if j is the last bin among those carrying items for o . In cases where an order o is only contained in a single bin ($|J_o| = 1$) the expected number of segments between the first and last occurrence of that order within its bin δ_{jo} results from the distribution above and is given by

$$\delta_{jo} = \begin{cases} \frac{(b_{jo}-1) \cdot (B_j+1)}{b_{jo}+1} + 1, & \text{if } j \in J_o \\ 0, & \text{if } j \notin J_o \end{cases} \quad \forall j \in J; o \in O. \quad (2)$$

Example 1. Consider a bin $j = 1$ containing $b_{12} = 2$ items dedicated to order $o = 2$ and $B_1 = 8$ items in total. If this bin is the initial bin containing items for order $o = 2$ in the release sequence, then the expected number of items occurring on the sorter once the first item dedicated to $o = 2$ passes by is $\gamma_{12} = \frac{2 \cdot 9}{3} = 6$. Consider another bin $j = 2$ with $b_{22} = 3$ and $B_2 = 5$. If this bin is the last among those carrying items for $o = 2$, then the expected number of sorter segments passing by until the last item for $o = 2$ is reached is $\gamma_{22} = \frac{3 \cdot 6}{4} = 4.5$. Consider another order $o = 3$, only contained in bin $j = 1$ with $b_{13} = 3$ items. The expected number of segments between first and last occurrence of order $o = 3$ is $\delta_{13} = \frac{2 \cdot 9}{4} + 1 = 5.5$ (see Fig. 2).

For any other bin in between an order o 's initial and last bin the order spread is prolonged by the complete bin content. Given a bin sequence ϕ , i.e., a permutation of wave J , with $\phi(j)$ and $\phi^{-1}(t)$ returning the sequence position of bin $j \in J$ and the bin at sequence position $t = 1, \dots, T$, respectively, the position of the initial bin of an order o is determined by $\alpha(o) = \min_{j \in J: b_{jo} > 0} \{\phi(j)\}$. The last sequence position of an order is given by $\omega(o) = \max_{j \in J: b_{jo} > 0} \{\phi(j)\}$. With these two sequence positions on hand, the order spread $s_o(\phi)$

of an order o amounts to

$$s_o(\phi) = \begin{cases} \gamma_{\phi^{-1}(\alpha(o)), o} + \sum_{t=\alpha(o)+1}^{\omega(o)-1} B_{\phi^{-1}(t)} + \gamma_{\phi^{-1}(\omega(o)), o}, & \text{if } \alpha(o) \neq \omega(o) \\ \delta_{\phi^{-1}(\alpha(o)), o}, & \text{if } \alpha(o) = \omega(o) \end{cases}, \quad \forall o \in O. \quad (3)$$

Consider the two bins $j = 1$ and $j = 2$ specified above being the first and last bin carrying items for order $o = 2$ in a sequence, respectively. In between these two bins is another bin $j = 3$ containing three items $B_3 = 3$. Irrespective of whether or not this bin contains items for $o = 2$ order spread s_2 is prolonged by the complete bin content and in total amounts to $s_2 = 6 + 3 + 4.5 = 13.5$. Moreover, the order spread of order $o = 3$, which is completely contained in bin $j = 1$, equals δ_{13} and, therefore, amounts to $s_3 = 5.5$ (see Fig. 2).

Among all permutations of bin set J our minimum order spread sequencing problem (MOSSP) seeks one permutation ϕ , which minimizes the total order spread over all orders $o \in O$, that is $\sum_{o \in O} s_o(\phi) \rightarrow \min$.

Due to the unpredictable induction process, our problem is of stochastic nature. However, by minimizing the expected order spreads we introduce a deterministic surrogate objective, whose appropriateness is tested in more detail in Section 4.3. In principle, minimizing the total order spread seems a well-suited objective, because of the following reason. Especially during peak hours the packing stations are often a bottleneck resource. An order occupies a packing station from the time the sorter directs the first item into the station. Then, the complete order is collected up to the arrival of the last item and, only then, a packing worker can finalize the customer order by packing it into a cardboard box. Thus, reducing the order spread from the first to the last occurrence of an order reduces the time a packing station is occupied and leaves more flexibility for the workforce of packing workers to timely release the lanes. This way, late packing processes and, thus, the probability that no lane is available for a newly arriving order can be reduced for a given workforce. Compared to a random release from the ASRS currently applied by our online retailer, in the long run, an optimized bin sequence may even allow to reduce the required workforce. The order spread is, thus, a surrogate objective for the actual objective. In the computational study of Section 4.3, we test its suitability in more detail with the help of a simulation study.

Example 2. Consider a given wave of $|J| = 6$ bins that contain partial orders of $|O| = 5$ customer orders. In Fig. 3 the number of items b_{jo} contained in a bin j dedicated to customer order $o \in O$ is given by the number in the shapes, i.e., circle, triangle, square, pentagon, and hexagon, each referring to a customer order. The bin number is given in the upper right edge of the respective bin and the dashed lines indicate the order spread of each order. The total sum of order spreads for alternative bin sequences (a) and (b) amounts to 46.58 and 57.58, respectively.

With regard to the computational complexity the following complexity status is to be attributed to MOSSP.

Theorem 1. The MOSSP is NP-hard in the strong sense.

Proof. See Appendix A. $\square$

3. Model formulation and solution procedures

This section presents algorithms for solving MOSSP. First, we introduce a mixed integer program, which can be fed into an off-the-shelf solver. Then, a simple priority rule-based procedure is presented, which allows us to gain solutions quickly. Afterwards, an exact dynamic programming approach and a heuristic beam search procedure are provided.

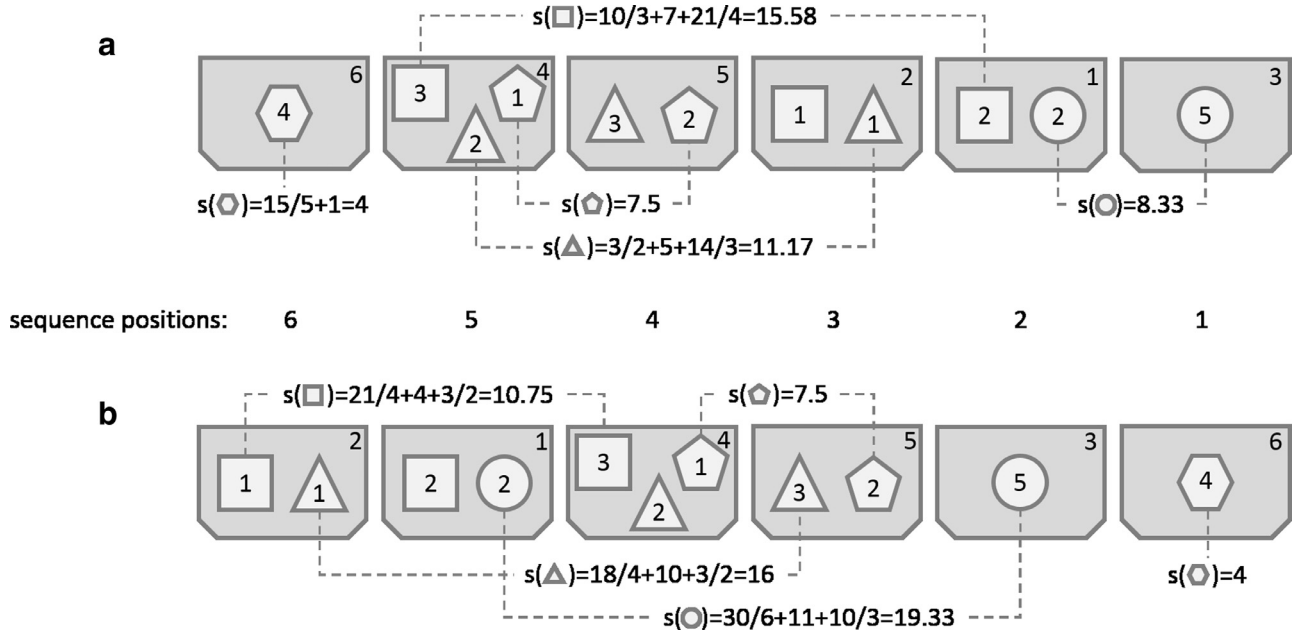


Fig. 3. Example.

Table 1
Notation.

J	Set of bins (index $j \in J$)
O	Set of customer orders (index $o \in O$)
J_o	Subset of bins containing items of order o
$T = J $	Number of sequence positions (indices $t, t_1, t_2 = 1, \dots, T$)
a_{jo}	Binary parameters: 1, an item for order o is contained in bin j ; 0, otherwise
B_j	Total number of items per bin
γ_{jo}	Expected number of sorter segments between the first item of order o in bin j to the last item in bin j resp. between the first item of bin j and the last item of order o in j
δ_{jo}	Expected number of sorter segments between the first and the last occurrence of an item of order o in bin j
M	Big integer
s_o	Continuous variables: spread of order o
x_{jt}	Binary variables: 1, if bin j is assigned sequence position t ; 0, otherwise

3.1. A mixed-integer program

First, we present a mixed integer program (MIP) that can be solved with a standard solver. Applying the notation summarized in Table 1 MOSSP can be formulated as a MIP (dubbed MOSSP-MIP) consisting of objective function (4) and constraints (5)–(9).

$$\text{MOSSP-MIP: Minimize } Z(X, S) = \sum_{o \in O} s_o \quad (4)$$

subject to

$$\sum_{j \in J} x_{jt} = 1 \quad \forall t = 1, \dots, T \quad (5)$$

$$\sum_{t=1}^T x_{jt} = 1 \quad \forall j \in J \quad (6)$$

$$s_o = \delta_{jo} \quad \forall o \in O; j \in J : J_o = \{j\} \quad (7)$$

$$s_o \geq \sum_{j \in J} \gamma_{jo} \cdot x_{j,t_1} + \sum_{t=t_1+1}^{t_2-1} \sum_{j \in J} B_j \cdot x_{j,t} + \sum_{j \in J} \gamma_{jo} \cdot x_{j,t_2} - M \cdot \left(2 - \sum_{j \in J} a_{jo} \cdot x_{j,t_1} - \sum_{j \in J} a_{jo} \cdot x_{j,t_2} \right) \quad (8)$$

$$\forall o \in O : |J_o| \geq 2; t_1 = 1, \dots, T-1; t_2 = t_1+1, \dots, T$$

$$x_{jt} \in \{0, 1\} \quad \forall j \in J; t = 1, \dots, T \quad (9)$$

Objective function (4) minimizes the total sum of order spreads. Constraints (5) and (6) ensure a well-defined sequence, that is each bin is assigned a sequence position and each sequence position a bin. Order spreads for orders only contained in a single bin are determined by (7) and inequalities (8) calculate order spreads for orders contained in at least two bins. Hereby, the bin spread is calculated by summing up the expected number of sorter segments between the first item of order o in the bin at sequence position t_1 to the last item of this bin (first sum), the number of items between the first and last bin containing items for order o (second

sum), and the expected number of sorter segments between the first item of the bin in sequence position t_2 to the last item of order o (third sum). These constraints are only tight, if the bins at positions t_1 and t_2 contain items of order o . Note, that big integer $M \geq \sum_{j \in J} B_j$ is sufficiently large. Finally, (9) defines the domain of the binary variables.

3.2. Priority rule-based procedure

In this section, we introduce a simple priority rule-based procedure, named CMOF (close many, open few), which successively assigns bins to sequence positions with the help of a simple myopic selection rule (see Algorithm 1).

Algorithm 1: Close-many-open-few procedure for MOSSP.

```

input: set of bins  $J$ , set of orders  $O$ , bin sets  $J_o$ 
1 initialize best sequence  $\pi^*$ 
2 foreach bin  $j$  in  $J$  do
    // fix the first bin of the sequence
3   create new (empty) sequence  $\pi$ 
4   fix  $j$  as first bin in sequence:  $\pi(1) = j$ 
5   remove  $j$  from  $J$ 
    // complete the bin sequence via priority rules
6   for sequence position  $p = 2$  to  $|J|$  do
        // determine next bin via priority rules
7       determine/update  $CL_i$  and  $OP_i$  for each remaining bin  $i$  in  $J$ 
8       schedule next bin  $\bar{i}$  via priority rule (a) resp. (b) at position  $p$ :
9        $\pi(p) = \bar{i}$ 
10      remove  $\bar{i}$  from  $J$ 
11  end
    // check for new best solution
12  if  $Z(\pi) < Z(\pi^*)$  then
13       $\pi^* = \pi$ 
14  end
return: best bin sequence  $\pi^*$ 

```

Thereby, we initially fix the first bin of our sequence π and determine the following two values for each remaining bin i :

- CL_i the number of orders closed, when sequencing i next. Completing an order means that, after i , none of the remaining bins will contain items of this order.
- OP_i the number of orders opened, when sequencing i next. Opening an order means that i is the first bin in our sequence containing items of this order.

We now pick the next bin in our sequence according to one of the following priority rules:

- (a) Bins with highest value CL_i first, 1st tie-breaker: lowest OP_i first, 2nd tie-breaker: lowest index first.
- (b) Bins with highest value $CL_i - OP_i$ first, tie-breaker: lowest index first.

This way, we iteratively assign bins to each position p of our sequence, until all bins are assigned and, then, determine the objective value $Z(\pi)$ of our sequence. We repeat this process with each bin in the first position to obtain multiple solutions and return the solution with minimal objective value $Z(\pi^*)$.

For our example introduced in Fig. 3 and a predefined first bin $j = 1$, the resulting bin sequence under both priority rules (a) and (b) is $1 \rightarrow 3 \rightarrow 6 \rightarrow 2 \rightarrow 4 \rightarrow 5$ with objective values 51.51. Note, however, that both priority rules leading to the same solution just exceptionally occurs (see Section 3.4).

3.3. Dynamic programming-based solution procedures

Next, we tackle MOSSP with a dynamic programming (DP) approach, which is an adaption of the elementary DP for sequenc-

ing problems provided by Held and Karp (1962). The decision process of DP is subdivided into $|J| + 1$ stages, where each stage $k = 1, \dots, |J|$ represents a sequence position (plus a virtual start stage 0). Each stage k contains states J' , with $J' \subseteq J$, representing the subset of bins already added to the first k sequence positions. The partial objective value already contributed by the fixed sequence positions is denoted by $f(J')$. There is a transition from state (J') to state (J'') if $J' \subset J''$ and $J'' \setminus J' = \{j\}$ with $j \in J$. The additional order spread $w(J', j)$ coming along with such a transition amounts to

$$w(J', j) = \sum_{o \in O} \begin{cases} \delta_{jo}, & \text{if } J_o = \{j\} \\ \gamma_{jo}, & \text{if } |J_o| \geq 2 \wedge J_o \cap J' \in \{\emptyset, J_o \setminus \{j\}\} \wedge j \in J_o \\ B_j, & \text{if } |J_o| \geq 2 \wedge J_o \cap J' \notin \{\emptyset, J_o \setminus \{j\}\} \\ 0, & \text{otherwise} \end{cases} \quad (10)$$

where J_o defines the subset of bins that contain items for customer order o . The first case handles single bin orders where all items are contained in current bin j . In the second case, the expected number of items passing by after (before) the first (last) item dedicated to order o is added, if bin j is the first (last) bin to start (complete) the collection of order o . In the third case, the contribution amounts to the complete bin content B_j , because collecting the order has already been started before j and will not be finished after j . Finally, the contribution is zero if collecting the order has not started yet or is already finished. Given the transitions' contribution the basic Bellman recursion is

$$f(J') = \min_{j \in J'} \{f(J' \setminus \{j\}) + w(J' \setminus \{j\}, j)\} \forall J' \subseteq J, J' \neq \emptyset \quad (11)$$

with $f(\emptyset) = 0$. We can determine $f(J)$ by a stage-wise forward recursion using (11). Finally, when last stage $k = |J|$ is reached and optimal solution value $f(J)$ is determined, a simple backward recursion can be applied to extract the optimum bin sequence.

We have $O(2^{|J|})$ states and $O(|J| \cdot 2^{|J|})$ transitions, which implies a computational effort of $O(|J| \cdot 2^{|J|})$. Note that the exponential growth in number of bins is in line with our complexity result.

To accelerate our DP, upper and lower bounds can be integrated. A global upper bound UB is determined upfront by some heuristic. Specifically, we apply our CMOF heuristic once with each priority rule (a) and (b) and utilize the better of the two solutions. Whenever a state specific lower bound $LB(J')$ plus the shortest path value $f(J')$ to current state J' is equal to or exceeds upper bound UB , the respective state can be fathomed as it cannot be part of a solution with a better objective value than the incumbent solution. Furthermore, we employ the CMOF heuristic with priority rule (b), which shows superior to rule (a) (see Section 3.4), to complete the best five nodes (according to $LB(J') + f(J')$) of each stage in order to tighten the upper bound. Our simple lower bound neglects the bin interdependency and assumes that all orders can be completed individually, so that the expected order spread of remaining orders contained in only a single bin (12), the minimum expected spread for orders already opened (13) and the minimum spread of orders not opened yet (14) sum up to

$$LB(J') = \sum_{\substack{o \in O: \\ |J_o| = 1}} \sum_{j \in J'} \delta_{jo} \quad (12)$$

$$+ \sum_{\substack{o \in O: \\ |J_o| \geq 2 \wedge J_o \cap J' \neq \emptyset}} \left(\sum_{j \in J'} B_j \cdot a_{jo} - \bar{B}_o^1(J') + \Gamma_o^1(J') \right) \quad (13)$$

$$+ \sum_{\substack{o \in O: \\ |J_o| \geq 2 \wedge J_o \cap J' = \emptyset}} \left(\sum_{j \in J'} B_j \cdot a_{jo} - \bar{B}_o^1(J') - \bar{B}_o^2(J') + \Gamma_o^1(J') + \Gamma_o^2(J') \right), \quad (14)$$

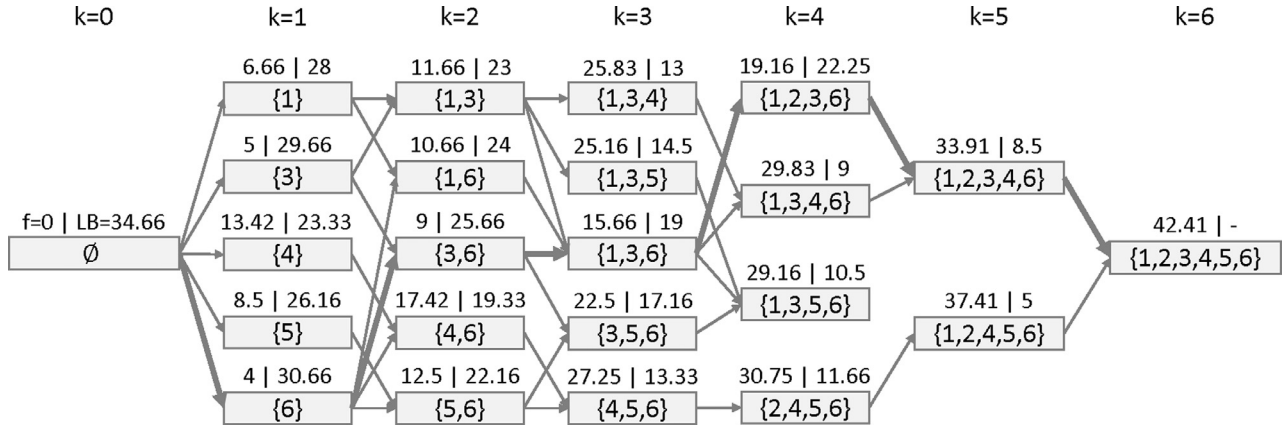


Fig. 4. Beam search graph for MOSSP.

Table 2
Parameters for instance generation.

Symbol	Description	Values	
		Small	Large
$ J $	Number of bins	5,6,...,20	30,35,40
$ O $	Number of orders	5,15,25	30,35,40
INT	Interval for order sizes	[1;5]	[1;5]
b	Bin spread	0.5	0.5

where $\bar{B}_o^1(J')$ and $\bar{B}_o^2(J')$ represent the two largest values from $(B_j \cdot a_{jo})_{j \in J'}$ with $\bar{B}_o^1(J') \geq \bar{B}_o^2(J')$, and $\Gamma_o^1(J')$ and $\Gamma_o^2(J')$ represent the two smallest values from $(\gamma_{jo})_{j \in J': a_{jo}=1}$ with $\Gamma_o^1(J') \leq \Gamma_o^2(J')$.

As the state space of our DP grows exponentially with the number of bins, we, furthermore, apply our DP scheme in a heuristic beam search approach. Beam search only branches the BW (beam width) most promising nodes per stage in the DP tree. For a sufficiently small BW, both space and runtime needed for solving MOSSP are polynomially bounded. However, there is no guarantee that the optimal solution is among those considered. To apply the BS heuristic, the BW best nodes per stage are selected according to $LB(J') + f(J')$. A pseudo code description of the BS procedure is given in [Appendix B](#).

The resulting bounded beam search graph for our example of [Fig. 3](#) with a given upper bound of $UB = 43$ and a beamwidth of $BW = 5$ is depicted in [Fig. 4](#). The final result is sequence $6 \rightarrow 3 \rightarrow 1 \rightarrow 2 \rightarrow 4 \rightarrow 5$ with an optimal objective value of 42.41. Note that to increase readability, only nodes are visualized that have not been discarded due to beamwidth or UB.

3.4. Computational performance

To show that our algorithms are well-suited for solving MOSSP we investigate their computational performance for differently sized problem instances. We distinguish two cases: small instances solvable to optimality by our DP approach and large instances representing real-world problems. [Table 2](#) displays the parameters used for instance generation.

For each combination of parameters instance generation is repeated 5 (small) and 3 times (large) resulting in a total of $5 \cdot 16 \cdot 3 \cdot 1 \cdot 1 = 240$ small and $3 \cdot 3 \cdot 3 \cdot 1 \cdot 1 = 27$ large instances, respectively. For a given number of bins $|J|$ and a given number of customer orders $|O|$ we generate the data as follows. For each order o we randomly draw the number of items η_o contained in this order

from the interval INT (with uniform distribution). Note that these small-sized orders are typical for our project partner and online retailing in general. Subsequently, we determine the number of bins $|J_o|$ containing order o : $|J_o| = 1 + b \cdot (\min\{\eta_o; |J| - 1\})$ (rounded to the nearest integer). This way, we ensure a feasible distribution of items between the minimum number of one bin and the maximum of $\min\{\eta_o; |J|\}$ bins. The bin spread parameter b allows us to control how strong orders are scattered amongst bins. The $|J_o|$ bins are randomly selected among all bins. Each of them receives one item and all remaining items are randomly distributed among these bins. Finally, we make sure that each bin contains at least one item; one randomly selected item of a randomly drawn order is added to each empty bin. Note that an empty bin may occur, if that bin was not (randomly) selected when distributing the items among all orders.

All procedures were implemented in Visual Basic and tests were performed on a 64-bit system on an Intel(R) Core(TM)2 Quad CPU with 2.83 gigahertz and 8 gigabytes memory.

First, we evaluate the performance of our beam search approach (BS) for the small instances whose optimal solutions can be obtained by our dynamic programming procedure (DP). For BS we apply a beamwidth of 1000, which, as previous tests have indicated, yields a good compromise between runtime and solution quality. Moreover, we apply optimization software ‘Gurobi’ (GB, version 7.0, see [Gurobi Optimization, 2016](#)) to compare our procedures with an off-the-shelf solver. Further results, obtained by drawing a random solution (RND) and by our myopic heuristics (CMOFa, CMOFb), are also added for means of comparison. Recall that random release sequences are currently applied by our online retailer and, therefore, RND values help us to compare our newly developed approaches with current practice. [Table 3](#) lists the average optimality gaps (in %) and the runtime (in CPU seconds) for these approaches. For reasons of clarity, we did not state the optimality gap for DP, which is always 0%.

Within a given time limit of 300 seconds, Gurobi was not able to find optimal solutions for every instance. The second column of [Table 3](#) lists the number of instances solved to optimality and the total number of instances for each parameter combination, respectively. In total, Gurobi is only able to solve 44% of the instances to optimality within the given time frame. Among the optimal approaches Gurobi is, thus, clearly outperformed by DP, which is able to solve all small instances to proven optimality. Note that increasing Gurobi’s time limit by a reasonable amount does not lead to significantly better solutions. When applying a time limit of 900 seconds, for instance (which seems quite long regarding the operational character of the problem), Gurobi can solve 55% of the in-

Table 3
Performance for small instances.

# bins	# opt	Optimality gap [%]					cpu time [seconds]		
		GB	RND	CMOFa	CMOFb	BS	GB	BS	DP
5	15/15	0.00	19.90	1.52	0.65	0.00	0.31	< 0.01	< 0.01
6	15/15	0.00	27.74	2.80	2.31	0.00	0.65	0.01	< 0.01
7	15/15	0.00	35.01	3.07	1.52	0.00	2.12	0.02	0.01
8	15/15	0.00	50.22	2.10	0.96	0.00	5.94	0.03	0.02
9	15/15	0.00	53.31	5.65	1.35	0.00	34.58	0.06	0.04
10	11/15	0.01	75.61	4.85	2.14	0.00	131.11	0.11	0.09
11	7/15	0.12	91.41	3.32	4.26	0.00	206.74	0.27	0.23
12	5/15	0.00	87.71	7.16	4.10	0.00	207.69	0.53	0.47
13	5/15	0.76	97.35	5.52	5.13	0.00	242.89	0.92	0.91
14	2/15	1.24	121.23	8.72	3.68	0.00	272.34	1.52	2.12
15	0/15	2.12	121.21	7.36	3.78	0.00	300.03	2.22	4.30
16	0/15	1.98	138.21	8.38	6.63	0.00	300.03	2.68	8.36
17	0/15	4.80	148.74	13.81	6.24	0.00	300.04	3.58	17.93
18	0/15	5.01	131.86	13.25	8.54	0.00	300.04	5.38	48.93
19	0/15	7.46	174.47	10.45	5.82	0.00	300.05	6.27	86.14
20	0/15	5.39	185.04	11.79	5.57	0.04	300.05	7.42	160.50
Avg		1.81	97.44	6.86	3.92	0.00	181.54	1.94	20.63
Max		41.05	282.60	52.23	21.60	0.54	300.09	11.90	553.83

Table 4
Performance for large instances.

I	Gap to GB [%]				cpu time [seconds]				
	RND	CMOFa	CMOFb	BS	RND	CMOFa	CMOFb	BS	GB
30	95.15	−15.38	−19.55	−30.99	< 0.01	0.04	0.04	54.92	900.22
35	85.90	−25.11	−37.03	−43.99	< 0.01	0.07	0.07	92.12	900.25
40	96.46	−33.81	−42.25	−50.31	< 0.01	0.10	0.10	137.50	900.61
Min	42.86	−48.42	−48.96	−58.43	< 0.01	0.04	0.04	47.20	900.11
Avg	92.50	−24.77	−32.94	−41.76	< 0.01	0.07	0.07	94.85	900.36
Max	149.95	15.43	2.48	−15.68	< 0.01	0.11	0.12	163.14	901.12

stances to optimality, whereby the average gap decreases by only 0.9%. At the same time the average cpu time per instance increases by almost 5 minutes. Thus, the gains of applying the DP approach hold good. However, the solution times of DP grow exponentially, which is in line with our complexity result and stresses the necessity to solve even larger instances heuristically. RND and CMOF are much faster options for solving MOSSP. Their solution time is barely measurable and, thus, not reported. In the worst case, however, these simple approaches lead to substantial optimality gaps of 283% (RND), 52% (CMOFa), and 22% (CMOFb). In order to deliver fast upper bounds especially CMOFb still seems to be quite useful with an average gap below 4%. The best compromise between solution quality and runtime is given by our BS approach, which finds an optimal solution in every instance except for one (i.e., the optimality gap of this instance amounts to 0.54%) within less than 2 seconds (12 seconds) of computational time on average (in the worst case).

These trends continue for larger instances as reported in Table 4. Within a given time frame of 900 seconds Gurobi (GB) is not able to find any optimal solution. Increasing the number of bins even further (≥ 45) leads to Gurobi not even finding feasible solutions within the time limit. The table displays the average gaps of random solutions (RND), myopic heuristics (CMOFa,b), and our beam search approach (BS) to the solution found by Gurobi as well as their runtimes. Gurobi is able to find better solutions than random ones (average gap of 93%), but is clearly outperformed by CMOF (25% with selection rule (a) and 33% with rule (b)) as well as BS (42%). BS (again executed with a beamwidth of 1000) finds the best solution among all competitors in each single instance within an acceptable runtime well below 3 minutes even for the largest instances.

It can be concluded that our beam search (BS) procedure seems well-suited to solve MOSSP even for instances of real-world size. Thus, for all subsequent tests we apply BS in the configuration defined above.

4. Simulation study

After deriving suited solution procedures for MOSSP, we are able to address some managerial aspects concerning the order consolidation process. In order to do so, we emulate the impact of different bin sequences in a simulation environment, whose setup will be described in detail in Section 4.1. Then, we aim to answer the question whether our surrogate objective (i.e., the order spread) is indeed a good proxy for the actual performance of the consolidation process (see Section 4.2). Furthermore, we evaluate the influence of several instance parameters and design alternatives in a sensitivity analysis (see Section 4.3).

4.1. Setup of simulation study

In this Section, we describe the setup of our simulation study, which emulates the consolidation process within a warehouse, to evaluate the actual performance impact of varying bin sequences released from the ASRS. Our prototype is the aforementioned warehouse of our online retailer. Fig. 5 depicts the general setup of the consolidation process. The main elements of the simulation to be defined are the layout of the consolidation conveyor, the items' movement along the conveyor, and the movement of the packing workers.

Conveyor layout: Starting from the induction station, we have a linear conveyor that moves along packing stations and ends in the leftover station. Items that cannot be properly unloaded, because

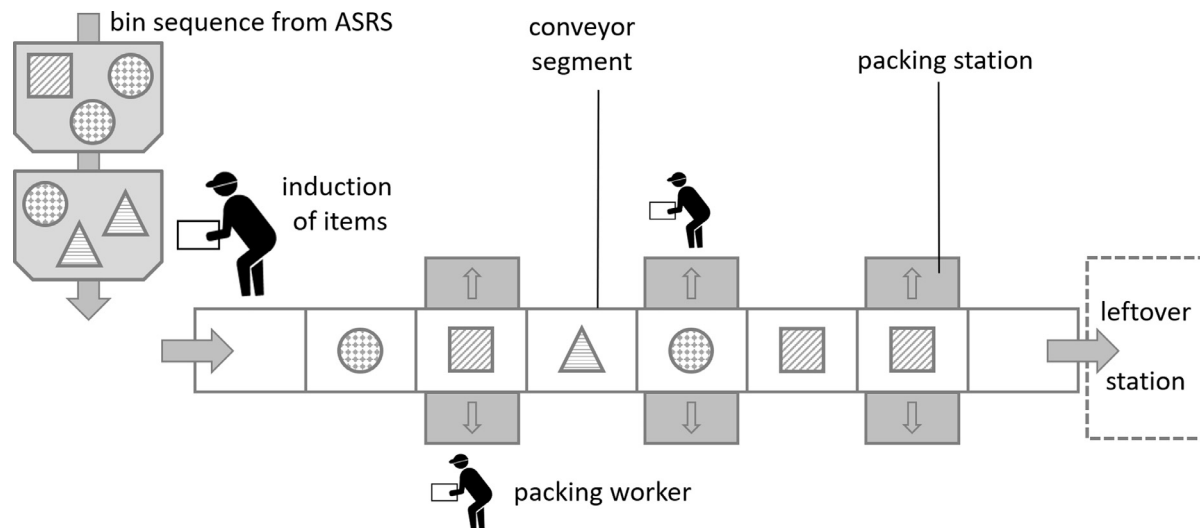


Fig. 5. General setup of the consolidation process.

all passed packing stations are blocked by other orders, are collected in the leftover station. These items are later manually added to the packed orders. This double handling is very time consuming and should, thus, be avoided. Note that a closed-loop conveyor is an alternative to a linear sorting conveyor. Once placed in the loop, items rotate until stations are available. Thus, closed-loop conveyors come by without a leftover area, but bear the risk of gridlocks (see Section 1.2). However, a linear conveyor is applied by our on-line retailer. The conveyor consists of equidistant line segments each having a size of 80 centimeters and carrying a single item. Starting in line segment three the given number of packing stations are positioned on every second segment towards both sides of the conveyor. Each station can receive any order so that size restrictions are a non-issue.

Item movement: Our simulation receives a bin sequence as an input. We simulate the induction process by retrieving a random item sequence from each bin and putting one item on each successive conveyor segment without empty segments in between. The items move one segment per time unit so that for a typical conveyor speed of 2.5 meters per second (see (Fedtke & Boysen, 2017)) the basic time unit of our simulation lasts 0.32 seconds. If the first item of an order reaches an empty station, this lane is assigned to the order, the item is removed from the conveyor and added to the packing station. Any subsequent item of the same order is added to the packing station whenever they pass by and once the last item of an order has reached the respective station, the order is marked as 'ready to be packed'. If all packing stations are blocked by started but not yet finished orders an item is finally added to the leftover area.

Worker movement: Initially all workers are placed at the leftover station and marked as 'available'. Once an order is in the status 'ready to be packed', the closest available worker is assigned to the respective packing station. Alternatively, a worker can become available right after packing another order and is, then, assigned the closest waiting order. For any worker movement towards an assigned packing station we randomly draw (with uniform distribution) his/her walking speed from interval [0.83; 1.39]. Thus, the walking speed ranges somewhere between 3 and 5 kilometers per hour, which is a typical velocity of human pickers in warehouses (see Schwerdfeger & Boysen, 2017). Once the packing station is reached, packing is assumed to just take a fixed time span of 5 seconds. This is the target time at our online retailer, which only includes folding the box, stowing items, pushing the

box onto another conveyor, and pressing a button to release the station. Adding packaging material, labeling, and closing the box is fully automated along the conveyor leading to the shipping area. We explore the impact of larger packing times in Section 4.3. Right after packing the worker is marked as 'available' and waits at the current position for a new assignment.

This way, we sample five random induction sequences, in which the items are withdrawn from the given bin sequence at the induction station, simulate the packing process, and record different performance criteria, which are finally averaged over the five different induction sequences. This seems necessary in order to reduce the impact of outliers/anomalies of random item sequences on the conveyor. Each bin sequence consists of three waves, where the first two waves are applied to initialize the process. Only for the final wave the performance criteria is measured. The relevant simulation time is started once the first item of the third wave is put onto the consolidation conveyor. Specifically, we consider three different performance measures, one for each elementary entity of the consolidation process, i.e., stations, orders, and workers:

- **Average number of occupied stations:** In each time unit (i.e., after each movement of the conveyor by one segment) we count the number of occupied stations currently accumulating an order and average this number over all time units until all orders are processed. The higher this number the larger the probability that items find no free packing station and, thus, reach the leftover station.
- **Fraction of completed orders:** Here, we calculate the percentage of all orders completely packed at a packing station so that no belonging item reaches the leftover area. To avoid the double-handling associated with leftover items a higher fraction indicates better performance. Note that in a closed-loop setup of the sorting conveyor blocked items rotate until finally being sorted so that the makespan for sorting all items varies. Therefore, the average number of completed orders per time unit seems an appropriate performance measure for these settings. However, in our linear setup the makespan is barely affected by the bin sequence, i.e., it is met once the final item reaches the end of the line or its dedicated lane. Only the fraction of completed orders is influenced by the order sequence and, therefore, seems the better performance indicator in our case.
- **The average packing time** is defined by the ratio of time units where a worker is packing items at stations to the full time

Table 5
Appropriateness of surrogate objective.

	J	O	Pearson			Avg. improvement [%]			Max. improvement [%]		
			w = 6	w = 8	w = 10	w = 6	w = 8	w = 10	w = 6	w = 8	w = 10
Occupied stations	30	30	0.69	0.73	0.73	16.77	16.98	17.85	24.31	23.95	25.87
	30	35	0.65	0.69	0.70	17.20	17.21	18.34	24.87	24.72	25.46
	30	40	0.68	0.69	0.70	16.33	16.08	16.81	23.50	23.19	24.66
	35	30	0.71	0.74	0.74	18.68	20.33	21.09	25.87	27.87	28.87
	35	35	0.71	0.72	0.72	18.09	19.05	19.93	25.52	27.08	28.04
	35	40	0.69	0.71	0.70	18.64	18.44	18.95	26.08	26.22	26.78
	40	30	0.72	0.76	0.76	20.79	23.33	24.24	28.69	31.32	32.78
	40	35	0.70	0.73	0.72	19.90	20.83	21.69	27.41	28.19	29.09
Completed orders	40	40	0.70	0.70	0.72	19.30	19.84	20.79	26.18	27.09	28.12
	30	30	−0.73	−0.76	−0.75	43.77	52.49	52.30	54.13	64.47	63.27
	30	35	−0.76	−0.77	−0.78	39.17	46.31	47.12	48.57	55.77	56.74
	30	40	−0.73	−0.76	−0.75	33.22	36.98	36.89	42.05	45.55	45.35
	35	30	−0.77	−0.79	−0.79	51.89	56.22	56.50	62.53	67.07	67.27
	35	35	−0.76	−0.78	−0.78	47.92	53.16	53.22	57.37	62.91	63.09
	35	40	−0.77	−0.78	−0.78	36.70	44.50	44.73	45.40	53.75	53.70
	40	30	−0.77	−0.79	−0.79	58.72	59.04	58.83	69.53	69.53	69.07
Packing time	40	35	−0.78	−0.79	−0.79	52.39	57.68	57.76	61.43	67.60	67.43
	40	40	−0.77	−0.80	−0.80	45.51	50.68	50.63	53.60	59.60	58.95
	30	30	−0.72	−0.69	−0.67	19.00	14.16	11.08	28.82	21.87	17.84
	30	35	−0.71	−0.67	−0.65	20.81	16.77	12.97	30.74	24.86	19.52
	30	40	−0.74	−0.69	−0.66	20.81	17.19	13.23	30.00	25.25	19.84
	35	30	−0.73	−0.68	−0.64	18.60	13.94	10.62	27.75	22.45	17.91
	35	35	−0.75	−0.69	−0.67	21.60	17.00	13.69	31.91	26.23	20.84
	35	40	−0.73	−0.70	−0.66	22.08	17.74	14.17	32.24	26.05	21.19
	40	30	−0.71	−0.67	−0.63	18.61	13.31	10.18	29.03	22.00	17.35
	40	35	−0.73	−0.68	−0.65	22.18	16.75	13.15	31.27	24.68	19.76
	40	40	−0.74	−0.68	−0.67	23.89	18.29	14.17	33.06	26.31	20.43

Table 6
Parameters for the sensitivity analysis.

Symbol	Description	Values	Default value
J	Number of bins	10,15,...,50	25
O	Number of orders	10,15,...,50	25
INT	Interval for order sizes	[1;3],[4;6],..., [28;30]	[1;5]
<i>b</i>	Bin spread	0.0, 0.1,..., 1.0	0.5
–	Conveyor speed [meters per second]	0.5,0.75,...,4.0	2.5
–	Packing time [seconds]	0.5,...,30	5
–	Number of stations	10,12,...,50	20
<i>w</i>	Number of packing workers	1,2,...,20	10

span of the simulation averaged over all packers of the given workforce. As a packer can only be in one of the three states ‘moving’, ‘packing’, and ‘waiting’, this measure defines the productive share of the workers’ service. Therefore, a higher value of this performance indicator is advantageous.

With this simulation setting on hand, we first address the appropriateness of our order spread objective.

4.2. Appropriateness of surrogate objective

Reducing the order spread is just a surrogate objective, which is supposed to indirectly support a fast accumulation of orders in packing stations so that finally a better performance of the consolidation process is enabled. This conjecture that the order spread is indeed a good proxy for the actual consolidation performance, which can, for instance, be measured by the three performance indicators presented in Section 4.1, is tested in this section. To do so, we measure the correlation between our surrogate objective, i.e., the total order spread, of randomly generated bin sequences and the simulation results for these sequences. Furthermore, we compare the performance indicators resulting from the simulation of an optimized bin sequence with those of random bin sequences.

Specifically, this test is structured as follows. We start our instance generator described in Section 3.4 with parameters $|J| \in \{30, 35, 40\}$ (number of bins), $|O| \in \{30, 35, 40\}$ (number of orders), $INT = [1; 5]$ (interval for order sizes), and $b = 0.5$ (bin spread) and repeat instance generation 10 times per parameter setting. This results in 90 different instances each consisting of three successive waves. For each of these instances we determine 100 random bin sequences (i.e., where three random wave sequences are concatenated) and a single optimized solution (i.e., where the three waves optimized with beam search follow each other). The consolidation process is simulated and the performance indicators defined in Section 4.1 are recorded for each of the 101 bin sequences. For the simulation we assume 10 packing stations and vary the number of packing workers w between 6, 8, and 10 workers. The results are summarized in Table 5. We report the Pearson product-moment correlation coefficient (Pearson) between our order spread objective and the respective performance indicator of the simulation for the different number of packing workers w . Note that a correlation coefficient of 1 (−1) shows that the two functions are a monotonically non-decreasing (non-increasing) transformation of each other, while a correlation coefficient of 0 signifies that the values of the two functions are independent of each other. Furthermore, we list the average and maximum improvement of the performance indi-

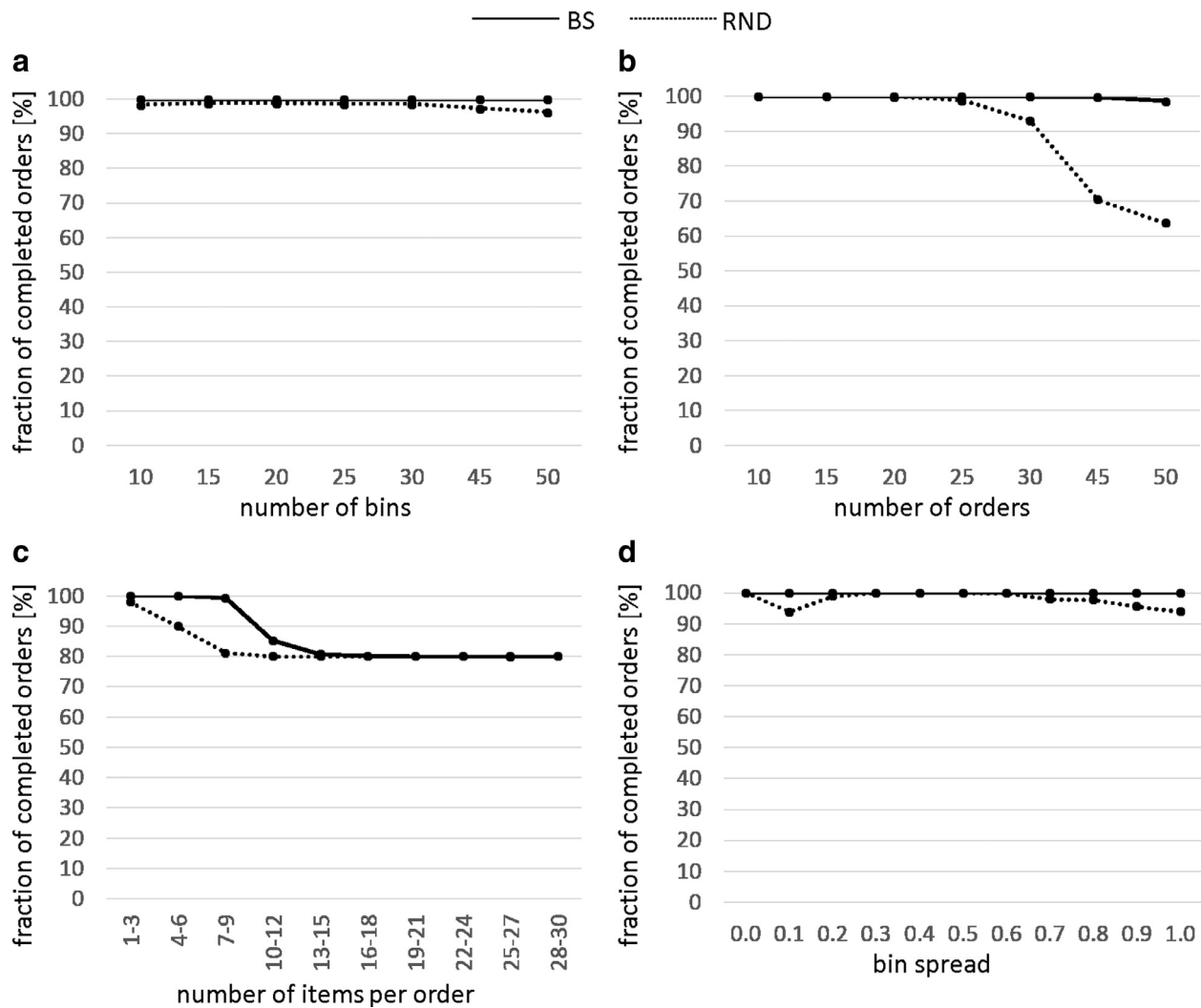


Fig. 6. Simulation results for varying parameter values of instance generation.

cator if the optimized bin sequence is fed into the simulation compared to the average random bin sequence.

The following conclusions can be drawn from the results summarized in Table 5:

- With regard to the correlation results it can be concluded that our surrogate objective successfully supports each of the three actual performance measures. The correlation to the average number of occupied stations is positive, which shows that a smaller total order spread reduces the number of occupied stations per time unit. The latter is desirable because this reduces the probability of items reaching the leftover area. The correlation to the fraction of completed orders is negative so that a lower order spread increases the number of completed orders. Finally, a smaller order spread raises the share of productive packing time (and therefore decreases unproductive moving and waiting time), which is indicated by the negative correlation between our surrogate objective and the average packing time.
- This positive result is further supported when comparing the performance indicators of bin sequences optimized according to the order spread and the average random bin sequence. Recall that random sequences are currently applied by our online retailer. Optimized bin sequences on average lead to between 16.08 and 24.24% less station occupation, between 33.22 and

59.04% more completed orders, and between 10.18 and 23.89% more productive packing time.

Thus, it can be concluded that our surrogate objective, i.e., the order spread, is indeed a well-suited objective. On the one hand, it results in a handy optimization problem, which does not try to model the complete consolidation process in detail. On the other hand, although being simple, it successfully supports a more efficient consolidation process.

4.3. On the impact of varying instance and system parameters

In this section, we explore how consolidation performance depends on variations of data instances (e.g., increasing order or bin number) and system parameters (e.g., conveyor speed or packing time). Such a sensitivity analysis aims to support warehouse managers having to set up an appropriate consolidation process. By measuring the increasing fraction of completed orders when speeding the conveyor up, for instance, warehouse managers can trade off these operational gains with increasing investment costs.

The parameter settings evaluated in our sensitivity analysis are summarized in Table 6. The upper four parameters are parameters handed over to our instance generator and we repeat instance generation 25 times. These parameters alter the characteristics of our data instances. Each resulting instance consists of 12 waves.

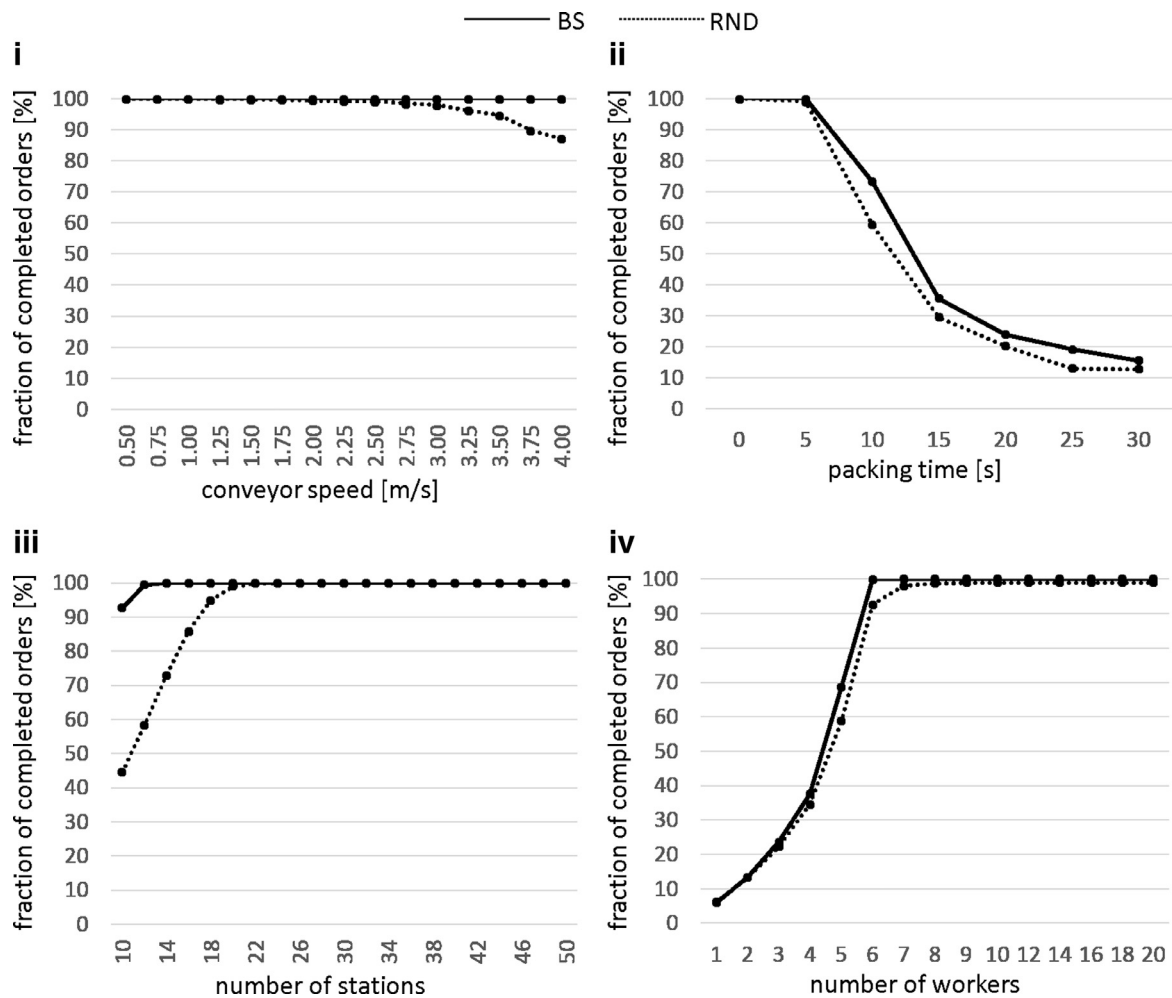


Fig. 7. Simulation results for various conveyor settings.

Two alternative bin sequences are derived for each instance: one concatenates wave sequences optimized with beam search and another consists of random wave sequences. For each bin sequence five different unloading sequences at the induction station are generated and the resulting consolidation process is simulated in our simulation environment. Performance measurement is started after wave 2. The bottom four parameters listed in Table 6 impact the setup of the consolidation process within our simulation. Their influence on the performance measures of the simulation is investigated as well.

Within our study the impact of each parameter listed in Table 6 is investigated separately. The simulation results for all parameter values of the current parameter are recorded while the values of all other parameters (currently not investigated) are set to their default value. First, Fig. 6 displays the results when varying the parameter values of instance generation. We compare the resulting simulation performance when varying the respective parameter for the bin sequence optimized with beam search (BS) and the random bin sequence (RND). For a matter clarity, we only report the fraction of completed orders, because the other two performance indicators show very similar results.

The following conclusions can be drawn from the simulation results depicted in Fig. 6:

- (a) In our setting, the number of bins a wave is collected in has (nearly) no impact on consolidation performance. Items are put on the conveyor segments at the induction station without empty segments in between, so that it barely matters in how

many bins a wave is conveyed towards induction. For our optimization approach it rather seems advantageous to have more bins, because then the bin sequence takes better control on the induction sequence. If a wave consists of just a few bins with large content, then an altered sequence has only negligible impact and induction leads to a more or less random item sequence on the conveyor anyway. The effect of the number of bins a wave of orders is distributed over is analyzed in more detail in the following section.

- (b) The performance of random bin sequences heavily reacts to an increasing order number. The more orders contained in the bins, the more important it is to quickly collect them in the packing stations. Obviously, optimized bin sequences are much more successful in this regard.
- (c) An increasing number of items per order also tends to increase the number of bins each order is contained in. The larger the latter, the longer it takes until orders are assembled. Optimized bin sequences show superior to lessen this effect. However, once each order consists of between 13 and 15 items, there are items per order in almost any bin. In such a setting even optimization can hardly impact the order spread anymore so that random bin sequences lead to identical performance.
- (d) The bin spread also influences the number of bins an order is contained in. Thus, at first sight, the same effect as above is expected here. However, with the number of items per order drawn from default interval $INT = [1; 5]$ an order can at most be spread among five bins so that there is still enough flexibility for MOSSP to considerably reduce order spreads.

Once valid forecasts for the above parameter values relevant to a specific warehouse setting are available, warehouse managers have to appropriately layout the consolidation area. Therefore, we also simulate the influence of different layout choices on consolidation performance (see Fig. 7):

- (i) The faster the conveyor, the higher the pressure to quickly consolidate all arriving items. In our simulation setting, an increasing number of uncompleted orders occurs for random bin sequences and very fast conveyors.
- (ii) System performance heavily reacts on larger packing times. Once the ambitious targets of our online retailer (i.e., 5 seconds) cannot be met, an ever increasing fraction of orders cannot successfully be completed. Thus, automating parts of the packing process (i.e., a computerized proposal for the right box sizes, fully-automated carton erection, adding of packing material, sealing, and labeling) should carefully be considered.
- (iii) As expected, more packing stations add consolidation capacity so that an increasing number of orders can successfully be completed. Again, random bin sequences are much more dependent on additional capacity.
- (iv) Analogously, increasing the number of packing workers leads to more completed orders. This effect holds true irrespective of the way bin sequences are determined. Once all orders can timely be packed, obviously, adding more workforce shows no effect on this performance measure.

It can be concluded that properly setting up a consolidation area is a delicate planning task, which is influenced by numerous instance and system parameters. Once packing is a bottleneck, optimizing the release sequences of bins from the ASRS according to the order spread objective considerably contributes to an efficient order consolidation.

Note that the order consolidation process is closely linked to the preceding picking process. Some further insights on the relationship of order picking and consolidation are provided in Appendix C.

5. Conclusions

This paper considers the minimum order spread sequencing problem. For a wave of picking orders collected under a batching and zoning picking policy and intermediately stored in an ASRS the release sequence of bins into the consolidation area is to be determined. In the consolidation area items are automatically sorted according to customer orders. The sorting process is executed by a consolidation conveyor where packing workers move from station to station in order to pack the collected orders into cardboard boxes. To quickly collect orders, so that packing workers have more time and flexibility during the consolidation process, our bin sequencing problem aims to reduce the order spread (i.e., the number of conveyor segments from an order's first occurrence up to the last) summarized over all orders. For the resulting problem computational complexity is proven and well-performing exact and heuristic solution procedures are introduced. Furthermore, we apply a simulation study of the consolidation process to confirm the appropriateness of our surrogate objective and investigate the impact of different layout alternatives.

Future research should compare the setup of the consolidation process investigated in this paper (i.e., with an intermediate ASRS and a linear consolidation conveyor) with alternative setups (i.e., intermediate storage vs. direct induction and a linear vs. closed-loop sorter). Since most online retailers structure their warehouses, such that they apply batching and zoning plus order consolidation, more decision support with regard to the best setup alternative for the consolidation process seems highly welcome.

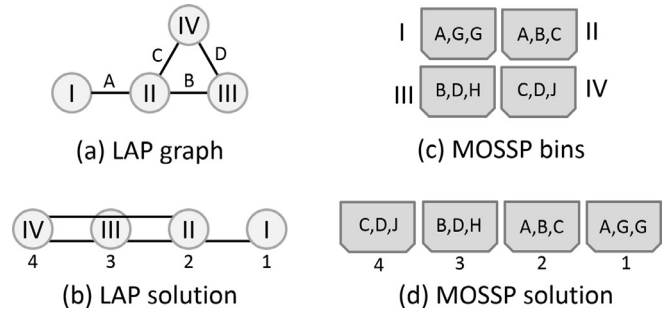


Fig. A.8. Example for the transformation between LAP and MOSSP.

Appendix A. Computational complexity of the minimum order spread sequencing problem

In this section, we show that the MOSSP is NP-hard. The proof is by transformation from the linear arrangement problem (LAP), which is well-known to be strongly NP-hard (see Garey & Johnson, 1979) and stated in the following:

LAP: Given a graph $G = (V, E)$ and a positive integer K . Is there a one-to-one-function $f: V \rightarrow \{1, 2, \dots, |V|\}$, i.e., a numbering of nodes V with integer values from 1 to $|V|$, such that

$$\sum_{(u,v) \in E} |f(u) - f(v)| \leq K?$$

The transformation scheme for generating an instance of MOSSP from a LAP instance is as follows. For each node of LAP we introduce a bin, i.e., $|J| = |V|$, so that we have a direct mapping between the numbering of nodes and the bin sequence. For each arc we introduce a customer order consisting of two items one each stored in the two bins representing the adjacent nodes. We dub any pair of these items and the orders they refer to as *edge items* and *edge orders*, respectively. Finally, to equalize the number of items contained in all bins we fill up all other bins until the fill level of the bin(s) representing the node(s) with the maximum degree are reached. Any bin is filled up with what we call *node items*, where all node items of a bin refer to the same customer order exclusively contained in the respective bin. The transformation and the corresponding solutions for an example are depicted in Fig. A.8.

Node items are only contained in a single bin so that the spread of the respective customer orders is not influenced by the bin sequence. Each bin contains an identical number of items B_j and there is only a single edge item per edge order. Thus, all γ_{jo} -values of edge items are identical, so that swapping initial and last bin containing a pair of edge items does not alter the objective value. The only aspect influencing the total order spread of a bin sequence are interfering bins stored in between any pair of bins containing edge items. As all bins contain an identical number of items B_j merely the total number of interfering bins has to be minimized. This number, however, directly corresponds to the absolute difference of the node numberings of LAP, which completes the proof.

Appendix B. Pseudo code of the Beam Search procedure

The following Algorithm 2 describes the BS procedure introduced in Section 3.3 via pseudo code.

Appendix C. On the relationship of order picking and consolidation

This section aims to provide deeper insights into the relation of the picking process and order consolidation. Specifically, we investigate the following question: Given a wave of orders to be col-

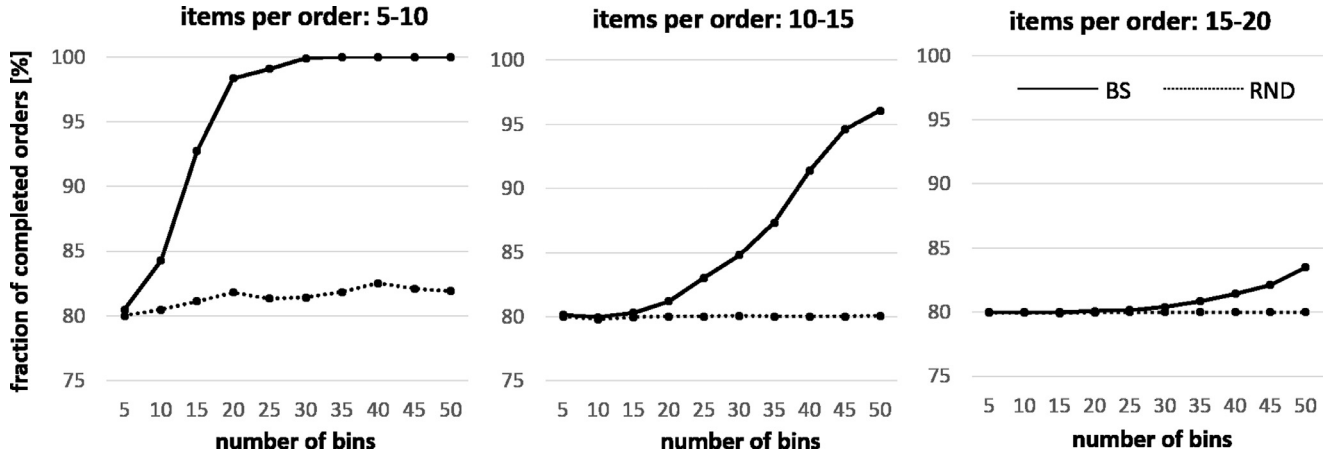


Fig. C.9. Simulation results if waves of orders are distributed among varying numbers of bins.

Algorithm 2: Beam search procedure.

```

input: set of bins  $J$ , set of orders  $O$ , bin sets  $J_0$ , upper bound solution  $\hat{\pi}$  with  $Z(\hat{\pi}) = UB$ , beamwidth  $BW$ 
1 initialize sets of nodes  $currentGen = \emptyset$  and  $nextGen = \emptyset$ 
2 add virtual starting node  $J^{init} = \emptyset$  to  $currentGen$ 
3 for stage  $k = 0$  to  $|J| - 1$  do
    // determine all nodes of successor stage  $k + 1$ 
    foreach node  $J'$  in  $currentGen$  do
        determine set  $CHILDREN$  of successors of node  $J'$ 
        foreach node  $J^{child}$  in  $CHILDREN$  do
            if  $f(J^{child}) + LB(J^{child}) < UB$  then
                if  $nextGen$  already contains  $J^{child}$  then
                    replace  $J^{child}$  in  $nextGen$  if partial objective value improves
                else
                    add  $J^{child}$  to  $nextGen$ 
            end
        end
    end
    // keep most promising nodes
    determine the  $BW$  best nodes in  $nextGen$  according to  $f(J') + LB(J')$ 
    remove all other nodes from  $nextGen$ 
     $currentGen = nextGen$ 
     $nextGen = \emptyset$ 
15 end
    // save best solution
16 if  $|currentGen| \geq 1$  then
17      $J^* =$  best node from  $currentGen$  according to  $f(J')$ 
18     extract bin sequence  $\pi^*$  from  $J^*$  via backward recursion
19 else
20     // no solution obtained that is better than  $UB$ 
21      $\pi^* = \hat{\pi}$ 
22 end
return: optimum bin sequence  $\pi^*$ 

```

lected, is it better to collect the items in fewer bins or is it advantageous to distribute the items among more bins? With regard to the picking process more bins means more pickers that collect items concurrently. A larger division of labor accelerates the parallel assembly of the wave, which is an important effect in times of tight delivery times, e.g., in e-business (Yaman et al., 2012). On the other hand, more pickers increase wage costs and reduce the number of items to be collected per picker and tour. The latter increases the frequency of returns to the depots so that additional (unproductive) walking results. With regard to order picking these effects are well-researched in the literature (e.g., see (de Koster et al., 2007; de Koster et al., 2012; Yu & de Koster, 2009)). We in-

vestigate the impact of the bin number on the consolidation process.

Specifically, we evaluate a set of 300 instances with an adjusted instance generation procedure. First, we generate a set of $|O| = 25$ orders with order sizes from the interval $INT \in \{[5; 10], [10; 15], [15; 20]\}$. The given order set is then distributed among different numbers of bins $|J| \in \{5, 10, 15, \dots, 50\}$ with a bin spread of $b = 0.5$ (see Section 3.4). This process is repeated 10 times for each parameter combination. Spreading the same order set among different numbers of bins $|J|$ is the main modification here. Subsequently, we determine a random sequence (RND) as well as an optimized sequence (i.e., determined with BS) and simulate the sorting process as described in Section 4.1 for each sequence. Hereby, we apply the default settings defined in Table 6.

The results illustrated in Fig. C.9 lead us to the following conclusions:

- If bins are released in random sequence from the ASRS, the number of bins a wave is distributed over does not (considerably) impact consolidation performance. The sequences of items passing the sorting conveyor are random irrespective of the bin number they arrive in.
- If we have very few bins, optimized bin sequences are barely better than random release sequences. In the very extreme where the complete wave is contained in a single bin, this bin is depleted in random sequence at the induction station so that a random sequence passes the sorting conveyor. With just a few bins there is little flexibility to counterbalance the random induction process.
- However, if a wave is spread over more bins, we receive a much better consolidation performance compared to random bin sequences. In the other extreme where each item is transported in its individual bin, optimization has most flexibility to reduce the order spread and leads to much better performance.
- The positive effect of additional bins heavily depends on the number of items per order. If there are more items, then it is harder for our optimization approach to minimize all order spreads, because there is so much overlap. Therefore, much more bins per wave are required to outperform random bin sequences if large orders are processed.

It can be concluded that an optimized release of bins greatly profits from more (and, thus, smaller) bins per wave. This leads to a much better consolidation performance. This effect has to be traded off against the negative effects of smaller bins per picker and tour during the picking process (see above).

References

- Bartholdi, J. J. III, & Hackman, S. T. (2014). Warehouse & distribution science. release 0.96. In *Supply chain and logistics institute*.
- Boysen, N., & Stephan, K. (2016). A survey on single crane scheduling in automated storage/retrieval systems. *European Journal of Operational Research*, 254, 691–704.
- Bozer, Y. A., Quiroz, M. A., & Sharp, G. P. (1988). An evaluation of alternative control strategies and design issues for automated order accumulation and sortation systems. *Material Flow*, 4, 265–282.
- Bozer, Y. A., & Sharp, G. P. (1985). An empirical evaluation of a general purpose automated order accumulation and sortation system used in batch picking. *Material Flow*, 2, 111–131.
- Briskorn, D., Emde, S., & Boysen, N. (2017). Scheduling shipments in closed-loop sortation conveyors. *Journal of Scheduling*, 20, 25–42.
- Fedtko, S., & Boysen, N. (2017). Layout planning of sortation conveyors in parcel distribution centers. *Transportation Science*, 51, 3–18.
- Gallien, J., & Weber, T. (2010). To wave or not to wave? order release policies for warehouses with an automated sorter. *Manufacturing & Service Operations Management*, 12, 642–662.
- Garey, M. R., & Johnson, D. S. (1979). *Computers and intractability: A guide to the theory of NP-completeness*. Freeman: New York.
- Gu, J., Goetschalckx, M., & McGinnis, L. F. (2007). Research on warehouse operation: A comprehensive review. *European Journal of Operational Research*, 177, 1–21.
- Gu, J., Goetschalckx, M., & McGinnis, L. F. (2010). Research on warehouse design and performance evaluation: A comprehensive review. *European Journal of Operational Research*, 203, 539–549.
- Gue, K. R., & Kim, B. S. (2007). Puzzle-based storage systems. *Naval Research Logistics*, 54, 556–567.
- Gurobi Optimization, I. (2016). *Gurobi optimizer reference manual*. <http://www.gurobi.com>.
- Held, M., & Karp, R. M. (1962). A dynamic programming approach to sequencing problems. *SIAM Journal*, 10, 196–210.
- Johnson, E., & Lofgren, T. (1994). Model decomposition speeds distribution center design. *Interfaces*, 24, 95–106.
- Johnson, E., & Meller, R. D. (2002). Performance analysis of split-case sorting systems. *Manufacturing Service Operations Management*, 4, 258–274.
- Johnson, M. E. (1998). The impact of sorting strategies on automated sortation system performance. *IIE Transactions*, 30, 67–77.
- de Koster, R., Le-Duc, T., & Roodbergen, K. J. (2007). Design and control of warehouse order picking: A literature review. *European Journal of Operational Research*, 182, 481–501.
- de Koster, R., Le-Duc, T., & Zaerpour, N. (2012). Determining the number of zones in a pick-and-sort order picking system. *International Journal of Production Research*, 50, 757–771.
- Litvak, N., & Vlasiov, M. (2010). A survey on performance analysis of warehouse carousel systems. *Statistica Neerlandica*, 64, 401–447.
- Meller, R. D. (1997). Optimal order-to-lane assignments in an order accumulation/sortation system. *IIE Transactions*, 29, 293–301.
- Owong, M., & Yih, Y. (2006). Picklist generation algorithm with order-consolidation consideration for split-case module-based fulfillment centres. *International Journal of Production Research*, 44, 4529–4550.
- Petersen, C. G. (2000). An evaluation of order picking policies for mail order companies. *Production and Operations Management*, 9, 319–335.
- Prokhorov, A. V. (2002). Negative hypergeometric distribution. In M. Hazewinkel (Ed.), *Encyclopaedia of mathematics*. Berlin: Springer-Verlag.
- Roodbergen, K. J., & Vis, I. F. A. (2009). A survey of literature on automated storage and retrieval systems. *European Journal of Operational Research*, 194, 343–362.
- Russell, M. L., & Meller, R. D. (2003). Cost and throughput modeling of manual and automated order fulfillment systems. *IIE Transactions*, 35, 589–603.
- Schwerdfeger, S., & Boysen, N. (2017). Order picking along a crane-supplied pick face: The SKU switching problem. *European Journal of Operational Research*, 260, 534–545.
- Weidinger, F., & Boysen, N. (2017). Scattered storage: How to distribute stock keeping units all around a mixed-shelves warehouse. *Transportation Science*. (in press).
- Yaman, H., Karasan, O. E., & Kara, B. Y. (2012). Release time scheduling and hub location for next-day delivery. *Operations Research*, 60, 906–917.
- Yu, M., & de Koster, R. (2009). The impact of order batching and picking area zoning on order picking system performance. *European Journal of Operational Research*, 198, 480–490.